



Workflow de développement - JobbingTrack

Vue d'ensemble

Ce guide décrit le workflow de développement complet pour contribuer à JobbingTrack, incluant la configuration, les tests, et le déploiement.

Configuration initiale

Prérequis système

```
# Vérifier les versions
node --version    # >= 18.0.0
npm --version     # >= 8.0.0
docker --version  # >= 20.0.0
git --version     # >= 2.30.0
```

Installation

```
# 1. Cloner le repository
git clone https://github.com/votre-org/jobbingtrack.git
cd jobbingtrack

# 2. Installation des dépendances
make install

# 3. Configuration des services
make up

# 4. Initialisation de la base de données
make init-all-dbs

# 5. Génération des données de test
make generate-test-data

# 6. Vérification de l'installation
make health
```

Structure du projet

```

jobbingtrack/
├─ backend/           # Services backend Node.js
|   ├─ api-gateway/   # API Gateway principal
|   ├─ auth-service/  # Service d'authentification
|   ├─ application-service/ # Gestion des candidatures
|   └─ ...            # Autres services
├─ frontend/         # Application Next.js
├─ tests/            # Tests complets
├─ scripts/          # Scripts utilitaires
├─ docs/             # Documentation
├─ makefiles/        # Makefiles modulaires
└─ docker-compose.yml # Configuration Docker

```

Approche recommandée : Utilisation des commandes Make

JobbingTrack utilise un système de **commandes Make** centralisées qui orchestrent le développement, les tests et le déploiement. Cette approche est **recommandée** car elle :

-  **Centralise** la logique de développement
-  **Gère** les dépendances automatiquement
-  **Configure** l'environnement (Docker, DB, etc.)
-  **Évite** les erreurs de configuration
-  **Standardise** les workflows

Commandes Make principales pour le développement

```

# Démarrage et développement
make up           # Services essentiels
make up-full      # Tous les services
make down         # Arrêt des services

# Tests et validation
make test-quick   # Tests rapides
make test-all    # Tests complets
make test-backend-only # Backend uniquement
make test-frontend-only # Frontend uniquement

# Qualité et diagnostic
make lint         # Linting (ESLint)
make format:check # Vérification formatage
make diagnostic   # Diagnostic complet
make logs         # Logs en temps réel

```

```
# Base de données
make init-all-dbs      # Initialisation DB
make generate-test-data # Données de test
make health            # Health check
```

Alternative : Commandes npm

Si les commandes Make ne sont pas disponibles ou pour un contrôle plus fin :

```
# Commandes npm équivalentes
npm run dev          # Développement
npm run test:*       # Tests spécifiques
npm run lint         # Linting
npm run format:check # Formatage
```

Note : Les exemples ci-dessous utilisent d'abord les commandes Make quand disponibles, puis les alternatives npm.

Workflow de développement

1. Création d'une branche

```
# Créer une branche feature
git checkout -b feature/nouvelle-fonctionnalite

# Ou une branche de correction
git checkout -b fix/bug-correction
```

2. Développement

Backend

```
# Démarrer les services
make up

# Travailler sur un service spécifique
cd backend/api-gateway
npm run dev
```

```
# Tests en cours de développement  
npm run test:watch
```

Frontend

```
# Démarrer le frontend  
cd frontend  
npm run dev  
  
# Tests en cours de développement  
npm run test:watch  
  
# Tests E2E en mode watch  
npm run test:e2e:ui
```

3. Tests et qualité

Tests automatiques

```
# Tests rapides (sans E2E)  
make test-quick  
  
# Tests complets  
make test-all  
  
# Vérification de la qualité  
npm run lint  
npm run format:check
```

Tests manuels

```
# Tests backend  
make test-backend-only  
  
# Tests frontend  
make test-frontend-only  
  
# Tests d'intégration  
make test-integration
```

4. Validation

Checklist avant commit

- ☐ Tests passent (`make test-quick` ou `make test-all`)
- ☐ Code linté (`make lint` ou `npm run lint`)
- ☐ Code formaté (`make format:check` ou `npm run format:check`)
- ☐ Coverage maintenu (>70%) (`make test-coverage`)
- ☐ Documentation mise à jour
- ☐ Tests d'accessibilité (si interface) (`make test-a11y`)

Pre-commit hooks

```
# Installation des hooks (si husky configuré)
npm run prepare

# Ou manuellement
cp scripts/pre-commit .git/hooks/
chmod +x .git/hooks/pre-commit
```

5. Commit et push

```
# Validation finale avec Make (recommandé)
make test-quick
make lint
make format:check

# Alternative npm si make non disponible :
npm run lint
npm run format:check

# Commit avec message descriptif
git add .
git commit -m "feat: ajouter authentication JWT

- Implémenter le service d'authentification
- Ajouter les routes de connexion/inscription
- Configurer la validation des tokens
- Tests unitaires et d'intégration"

# Push vers la branche
git push origin feature/nouvelle-fonctionnalite
```

6. Pull Request

Template PR

Description

[Description claire de la fonctionnalité/correction]

Type de changement

- [] Bug fix (correction non-breaking)
- [] New feature (ajout fonctionnel)
- [] Breaking change (changement cassant)
- [] Documentation
- [] Tests

Tests

- [] Tests unitaires ajoutés
- [] Tests d'intégration ajoutés
- [] Tests E2E mis à jour
- [] Coverage maintenu >70%

Checklist

- [] Code review par au moins 2 développeurs
- [] Tests CI/CD passent
- [] Documentation mise à jour
- [] Variables d'environnement documentées
- [] Tests d'accessibilité (si interface)

Environnements de développement

Développement local

```
# Services essentiels
make up

# Tous les services
make up-full

# Avec monitoring
make up-profile PROFILE=monitoring
```

Base de données

```
# Développement
make up-dev
# URL: postgresql://localhost:5433/jobbingtrack

# Tests
make up-test
# URL: postgresql://localhost:5434/jobbingtrack_test
```

Services disponibles

```
# Frontend
http://localhost:8080

# API Gateway
http://localhost:3000

# Services backend
http://localhost:3001 # Auth
http://localhost:3002 # Applications
http://localhost:3007 # Dashboard

# Monitoring
http://localhost:9090 # Prometheus
http://localhost:3000/metrics # Métriques
```

Outils de développement

Debugging

Backend

```
// Ajouter des logs de debug
console.log('Debug info:', data);

// Utiliser le debugger
debugger;

// Logs structurés
logger.info('Information message');
```

```
logger.warn('Warning message');  
logger.error('Error message');
```

Frontend

```
// React DevTools  
// Console du navigateur  
  
// Logs Redux/Zustand  
console.log('State:', store.getState());  
  
// Network tab pour les API calls
```

Monitoring

```
# Logs en temps réel  
make logs  
  
# Status des services  
make status  
  
# Health check  
make health  
  
# Métriques  
make metrics
```

Performance

```
# Profile des performances  
make up-profile PROFILE=monitoring  
  
# Tests de performance  
make test-performance  
  
# Analyse des métriques  
# Prometheus: http://localhost:9090  
# cAdvisor: http://localhost:8081
```


Intégration continue

Pipeline automatique

1. **Security Scan** - Audit des vulnérabilités
2. **Code Quality** - ESLint, Prettier, TypeScript
3. **Backend Tests** - Tests unitaires et d'intégration
4. **Frontend Tests** - Tests unitaires et E2E
5. **System Integration** - Tests de santé et d'intégration
6. **Reporting** - Artefacts et notifications

Artefacts générés

```
ci-artifacts/  
├─ summary.json           # Résumé de la pipeline  
├─ backend/coverage/     # Coverage backend  
├─ frontend/coverage/    # Coverage frontend  
├─ test-results.xml      # Résultats JUnit  
└─ security/audit.json   # Rapport de sécurité
```

Déploiement

Environnements

```
# Staging  
make up-staging  
  
# Production (simulation)  
make up-prod  
  
# Tous les environnements  
make up-all-dbs
```

Variables d'environnement

```
# .env.example  
NODE_ENV=development  
DATABASE_URL=postgresql://localhost:5432/jobbingtrack
```

```
REDIS_URL=redis://localhost:6379
JWT_SECRET=your-secret-key
```

Build de production

```
# Build backend
make build

# Build frontend
cd frontend && npm run build

# Tests de production
make test-all
```

Bonnes pratiques

Code

1. Conventions de nommage

```
// ✅ Bon
const userProfile = await getUserProfile(userId);
const isUserAuthenticated = checkAuthentication();

// ❌ Éviter
const x = await getData(id);
const flag = checkAuth();
```

2. Structure des fonctions

```
// ✅ Bon
async function createUser(userData) {
  // Validation
  validateUserData(userData);

  // Business logic
  const user = await User.create(userData);

  // Events
  await publishUserCreatedEvent(user);
}
```

```
    return user;
}
```

3. Gestion d'erreurs

```
//  Bon
try {
  const result = await riskyOperation();
  return result;
} catch (error) {
  logger.error('Operation failed', { error, context });
  throw new CustomError('Operation failed', error.code);
}
```

Tests

1. Coverage

- Minimum 70% global
- Tester les cas d'erreur
- Tests d'intégration pour les API

2. Noms de tests

```
//  Bon
test('should authenticate user with valid credentials', async () => { ... });
test('should reject authentication with invalid password', async () => { ... });

//  Éviter
test('should work', async () => { ... });
test('test login', async () => { ... });
```

3. Structure AAA

```
test('should create user', async () => {
  // Arrange
  const userData = { email: 'test@example.com' };

  // Act
  const response = await request(app)
    .post('/api/users')
    .send(userData);

  // Assert
```

```
expect(response.status).toBe(201);  
expect(response.body).toHaveProperty('id');  
});
```

Git

1. Commits atomiques

```
# ✅ Bon  
git commit -m "feat: add user authentication  
  
- Implement JWT authentication  
- Add login/logout endpoints  
- Update user model with auth fields  
- Add comprehensive tests"  
  
# ❌ Éviter  
git commit -m "update stuff"  
git commit -m "fix"
```

2. Branches thématiques

```
# ✅ Bon  
feature/user-authentication  
fix/login-validation  
refactor/api-cleanup  
  
# ❌ Éviter  
patch-1  
update  
fix-bug
```

Troubleshooting

Problèmes courants

1. Tests qui échouent

```
# Diagnostic complet  
make diagnostic
```

```
# Logs détaillés
make logs

# Status des services
make status

# Reset de la base de données
make db-reset
```

2. Services qui ne démarrent pas

```
# Nettoyage complet
make clean-force

# Redémarrage
make up-full

# Vérification des ports
make diagnostic-network
```

3. Problèmes de cache

```
# Nettoyer le cache Docker
make clean-docker-cache

# Nettoyer npm cache
npm cache clean --force

# Rebuild des services
make rebuild
```

Logs et debugging

```
# Logs de tous les services
make logs

# Logs d'un service spécifique
make logs-service SERVICE=api-gateway

# Logs des tests
npm run test 2>&1 | tee test.log
```

```
# Debug avec breakpoints  
node --inspect src/server.js
```

Performance

```
# Monitoring des performances  
make up-profile PROFILE=monitoring  
  
# Tests de performance  
make test-performance  
  
# Health check système  
make health
```

Ressources

Documentation

- [Architecture Guide](#)
- [API Documentation](#)
- [Testing Guide](#)
- [CI/CD Pipeline](#)
- [Development Guide](#)

Outils

- [GitHub Issues](#)
- [GitHub Projects](#)
- [Discord/Slack](#)

Support

```
# Commandes d'aide  
make help  
make help-up  
make help-test  
  
# Documentation des scripts  
./scripts/README.md
```

```
# FAQ
docs/FAQ.md
```

Standards de code

JavaScript/TypeScript

```
// ✅ Respecting ESLint rules
const { userId, userName } = user;
const formattedName = userName.toLowerCase();

// ✅ Gestion d'erreurs
async function getUser(id) {
  try {
    const user = await User.findById(id);
    if (!user) {
      throw new NotFoundError('User not found');
    }
    return user;
  } catch (error) {
    logger.error('Failed to get user', { id, error });
    throw error;
  }
}
```

React/Next.js

```
// ✅ Hooks personnalisés
function useUserProfile(userId) {
  return useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUserProfile(userId),
    enabled: !!userId,
  });
}

// ✅ Composants typés
interface UserProfileProps {
  user: User;
  onEdit: (user: User) => void;
}

function UserProfile({ user, onEdit }: UserProfileProps) {
```

```

return (
  <div className="user-profile">
    <h2>{user.name}</h2>
    <button onClick={() => onEdit(user)}>
      Edit Profile
    </button>
  </div>
);
}

```

Tests

```

// ✅ Tests descriptifs
describe('UserService', () => {
  describe('createUser', () => {
    test('should create user successfully', async () => {
      // Arrange
      const userData = { email: 'test@example.com', name: 'Test User' };

      // Act
      const result = await userService.createUser(userData);

      // Assert
      expect(result).toHaveProperty('id');
      expect(result.email).toBe(userData.email);
    });

    test('should validate required fields', async () => {
      // Arrange
      const invalidData = { email: 'invalid-email' };

      // Act & Assert
      await expect(userService.createUser(invalidData))
        .rejects.toThrow('Name is required');
    });
  });
});

```

Contribution

Processus

1. **Créer une issue** pour toute nouvelle fonctionnalité/bug
2. **Discuter** avec l'équipe avant de commencer
3. **Créer une branche** depuis `develop`

4. **Développer** avec tests et documentation
5. **Tester** localement et en CI/CD
6. **Créer une PR** avec description détaillée
7. **Code review** par au moins 2 développeurs
8. **Merge** après approbation

Critères d'acceptation

- ☐ Tests passent (CI/CD vert)
- ☐ Code review approuvé
- ☐ Documentation mise à jour
- ☐ Tests d'accessibilité (si interface)
- ☐ Performance validée
- ☐ Sécurité vérifiée

Ce workflow évolue avec le projet. Consultez régulièrement les mises à jour et proposez des améliorations.